

MACAU UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Computer Science and Engineering

Faculty of Innovation Engineering

<<Software Project for Course Software Engineering>>

Homework ID : Task3-Requirements Analysis

Report Title : TrajVis Studio: Requirements Analysis Specification

Group Number : 1

Team Members: Chen Hongzhou 1240007950 (team leader)

Zhou Guoxi 1240007947

Xu Jing 1240007919

Date :

[TO BE CONFIRMED: submission date]

Abstract

This document is the requirements analysis for **TrajVis Studio**, the system proposed in Task 1 and specified in Task 2. It models the system using object-oriented analysis, which is one of the two routes Part A of the task description permits.

The analysis is presented in the three stages the task description prescribes for the object-oriented route. **Functional modelling** (Chapter 2) fixes the boundary of the system by naming its actors and its use cases. **Static modelling** (Chapter 3) identifies the classes the system is made of, their attributes, their operations and the relationships between them. **Dynamic modelling** (Chapter 4) traces, for every use case, the sequence of messages that carries it out. Chapter 5 adds a state diagram for the one object whose behaviour is genuinely state-dependent, which Part A welcomes as an extra.

Every model element is traced back to the requirement it satisfies, so that the analysis can be checked against Task 2 one item at a time. Content that depends on the customer assigned under Task 2 Part A source 1, or on the elicitation that is still outstanding, is printed in red **like this** and is not invented.

Table of Contents

Abstract II

Table of Contents IV

List of Figures V

List of Tables VI

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 Notation . . . 1
- 1.3 Scope . . . 2
  - 1.3.1 In scope . . . 2
  - 1.3.2 Out of scope . . . 2
- 1.4 Relation to the Earlier Deliverables . . . 2
- 1.5 Contributions . . . 2
- 1.6 Overview . . . 2

2 Functional Modelling: Use Cases 3
- 2.1 Actors . . . 3
- 2.2 Use Case Diagram . . . 3
- 2.3 Use Case Descriptions . . . 4
  - 2.3.1 UC-1: Ask a Question in Ordinary Language . . . 5
  - 2.3.2 UC-2: Retrieve and Filter Trajectories . . . 5
  - 2.3.3 UC-3: Select and Draw the Figure . . . 6
  - 2.3.4 UC-4: Summarise the Result in Words . . . 6
  - 2.3.5 UC-5: Show the Accuracy Dashboard . . . 7
  - 2.3.6 UC-6: Record and Re-run an Exchange . . . 7
- 2.4 Traceability to the Requirements . . . 8

3 Static Modelling: Class and Object Diagrams 9
- 3.1 Finding the Classes . . . 9
- 3.2 Class Diagram . . . 10
- 3.3 Relationships . . . 12
- 3.4 Object Diagram . . . 12
- 3.5 Traceability to the Requirements . . . 14

4 Dynamic Modelling: Sequence Diagrams 15
- 4.1 Consistency with the Class Diagram . . . 19

5 Behavioural Modelling: State Diagram **20**

References **22**

List of Figures

2-1 Use case diagram. 4

3-1 Class diagram, part 1: the data classes. 11
3-2 Class diagram, part 2: the question-and-answer classes. 11
3-3 Object diagram for one answered exchange. 13

4-1 Sequence diagram for UC-1. 15
4-2 Sequence diagram for UC-2. 16
4-3 Sequence diagram for UC-3. 16
4-4 Sequence diagram for UC-4. 17
4-5 Sequence diagram for UC-5. 18
4-6 Sequence diagram for UC-6. 19

5-1 State diagram for an exchange. 21

List of Tables

2-1 Actors and the role each plays. 3
2-2 Every functional requirement of Task 2 is carried by exactly one use case. . . . 8

3-1 Candidate classes and the reason each is kept or dropped. 10
3-2 Relationships between the classes, read from Figures 3-1 and 3-2. 12
3-3 Classes, and the requirements they carry out. 14

Chapter 1

Introduction

1.1 Purpose

This document is the Requirements Analysis Specification (RAS) for **TrajVis Studio**, prepared by Group 1 for Task 3. Task 2 said *what* the system must do; this document says *how the system is put together* to do it.

It serves three readers. The **build team** takes the class diagram and the sequence diagrams as the design it implements against, so that Task 4 and the code that follows are a continuation of this document rather than a fresh start. The **assigned customer** sees the use cases as the agreed boundary of the system and can object to a use case that is missing or wrongly scoped before any code exists. The **course assessors** check that every model element can be traced to a stated requirement.

1.2 Notation

All diagrams in this document use the Unified Modelling Language (UML), and all of them were drawn with a diagram tool rather than by hand, as Part B(2) of the task description requires. In addition:

- Every use case carries an identifier `UC-`n and every class an identifier of the form `C-`n. Requirements keep the identifiers they were given in Task 2, `REQ-1.1` to `REQ-1.6`.
- In the class diagram, an association end marked with a filled diamond is a composition and one marked with a hollow diamond is an aggregation; an unmarked end is a plain association.
- In the sequence diagrams, a solid arrow is a synchronous call, a dashed arrow is a return, and a bar on the lifeline is the period during which the participant is active.
- **[TO BE CONFIRMED: one line explaining any other notational convention used later in the document, if any is introduced]**

1.3 Scope

1.3.1 In scope

The analysis covers the whole round trip the system performs: understanding a question written in ordinary language, retrieving and filtering the records the question refers to, choosing and drawing the figure, describing the result in words, recording the exchange so it can be re-run, and measuring the system's own accuracy against a recorded benchmark.

1.3.2 Out of scope

The analysis does not cover the collection or the cleaning of the movement data, since the system reads records that already exist. It does not cover planning decisions, a mobile client or a public web service, all of which Task 2 placed out of scope and which therefore have no use case here.

1.4 Relation to the Earlier Deliverables

Task 1 proposed the project and Task 2 stated the requirements; this document is the third and last step before design and coding. The mapping is direct: the six functional requirements `REQ-1.1` to `REQ-1.6`, which came from features F1 to F6 of Task 1, become the six use cases `UC-1` to `UC-6` of Chapter 2. Each of those use cases is then realised by the classes of Chapter 3 and by one sequence diagram in Chapter 4.

Task 2 left 78 items to be confirmed, most of them waiting on the customer assigned under Part A source 1 and on the elicitation that has not yet been carried out. This analysis does not wait for them: the parts of the model that are independent of the customer are complete, and the parts that are not are marked in red and left open.

1.5 Contributions

[TO BE CONFIRMED: name, for each of the three members, what that member contributed to this report. The division has to be settled first; do not write it by guessing. Once the diagrams are drawn and the use cases written, the split is a real one: two members can take three use cases each, with the class diagram, the sequence diagrams and the write-up divided along the same lines as Table ??.]

1.6 Overview

Chapter 2 names the actors and the use cases and describes each use case in full. Chapter 3 identifies the classes and presents the class diagram and an object diagram. Chapter 4 gives a sequence diagram for every use case, and Chapter 5 adds a state diagram.

Chapter 2

Functional Modelling: Use Cases

2.1 Actors

An actor is a role played by a person or by another system that interacts with the system being analysed. The three human roles are the ones Task 2 already identified in its Table 2-1; the two system roles are the sources the system reads.

Table 2-1: Actors and the role each plays.

Actor	Kind	Role
Analyst	Human, primary	A person with a question about recorded movement who cannot write a data query. Starts almost every use case. *(Source: Task 2, Table 2-1)*
Reviewer or auditor	Human, secondary	Checks how a past answer was produced and re-runs it.
Assigned customer	Human, secondary	The team assigned by the Moodle group table; confirms that the analysis matches the requirement. *(Source: Task 2, Part A source 1)*
Trajectory data store	System	Holds the recorded movement the system reads. Never written to.
Recorded benchmark	System	Holds published question-and-answer pairs used to measure the system's own accuracy.

[TO BE CONFIRMED: whether the assigned customer team introduces an actor of its own — the actor list cannot be closed until Part A source 1 has been elicited]

2.2 Use Case Diagram

Figure 2-1 shows the boundary of the system and the use cases it offers. The six use cases correspond one to one with the six functional requirements of Task 2, so the diagram can be read against Task 2 without a translation table.

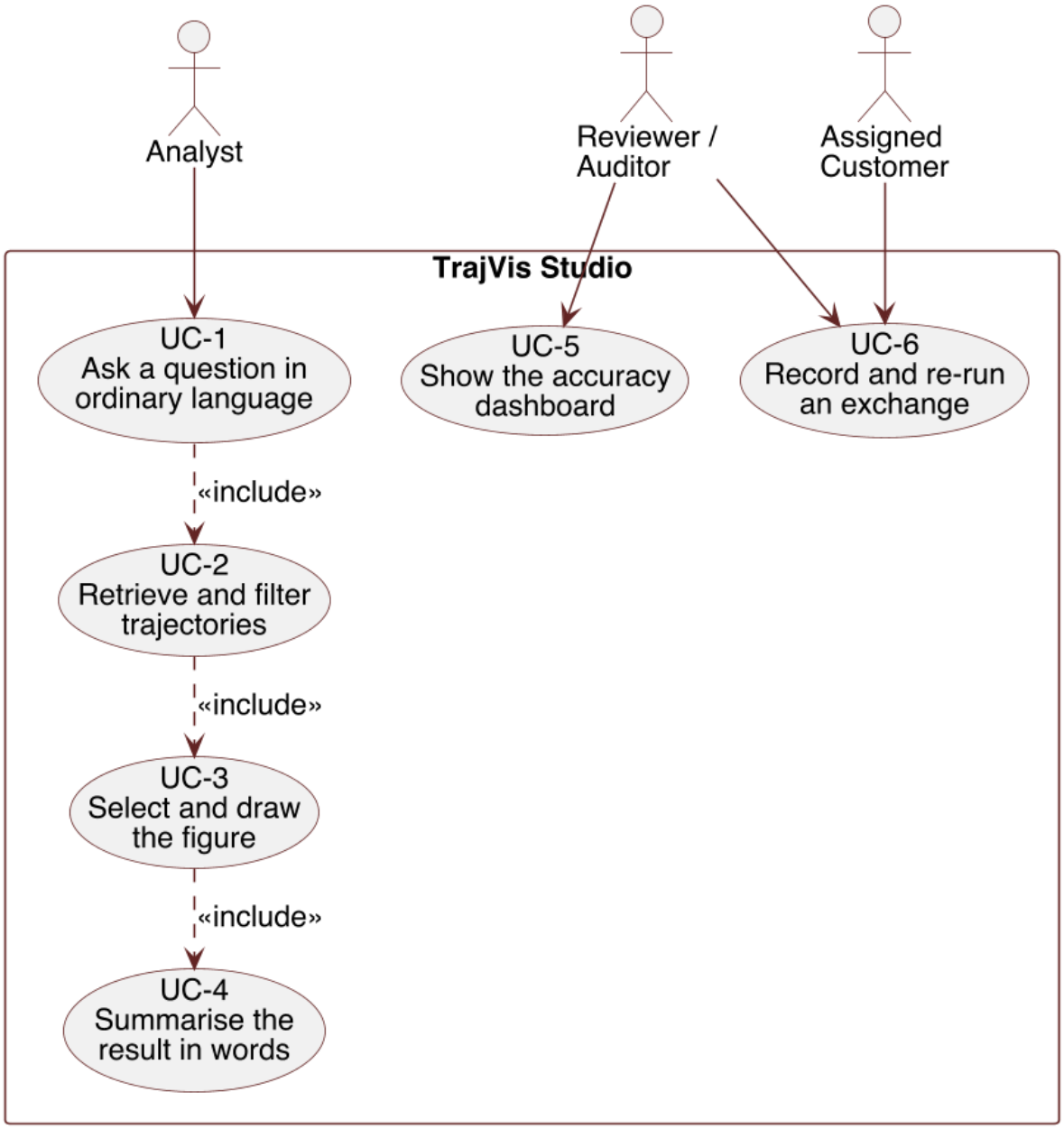


Figure 2-1: Use case diagram.

Four of the six use cases are refinements of one another rather than independent goals, and the diagram records this with `include`: a question cannot be answered without first being interpreted, and a figure cannot be drawn without first retrieving the records. `UC-5` and `UC-6` stand on their own, because the accuracy dashboard and the re-run of a past exchange can each be used without asking a new question.

2.3 Use Case Descriptions

Each use case is described below by the same seven fields, so that the descriptions can be compared with one another and, later, with the sequence diagrams of Chapter 4.

2.3.1 UC-1: Ask a Question in Ordinary Language

Actor	Analyst
Goal	Obtain an exact written request from a question typed in ordinary language, without writing a query.
Preconditions	The analyst has opened the system. The trajectories of interest are already held in the data store.
Main success scenario	**[TO BE CONFIRMED: the numbered steps, once the format has been agreed with the customer. Draft from Task 2 REQ-1.1 and Task 1 §3.4, feature F1.]**
Alternative flows	**[TO BE CONFIRMED: what happens when the question cannot be interpreted at all]**
Postconditions	The derived request is held for the rest of the session and shown back to the analyst beside the question.
Traces to	REQ-1.1

2.3.2 UC-2: Retrieve and Filter Trajectories

Actor	System (on behalf of the Analyst)
Goal	Find the trajectories the written request refers to and return only the matching subset.
Preconditions	A written request exists.
Main success scenario	**[TO BE CONFIRMED: the numbered steps. Draft from Task 2 REQ-1.2 and Task 1 §3.4, feature F2.]**
Alternative flows	**[TO BE CONFIRMED: what happens when no record matches, and when a place name in the request cannot be resolved]**
Postconditions	A subset of the records is selected and identified by a snapshot identifier.
Traces to	REQ-1.2

2.3.3 UC-3: Select and Draw the Figure

Actor	System (on behalf of the Analyst)
Goal	Choose the chart that fits the question and render it, with no chart type or plotting tool chosen by the analyst.
Preconditions	A subset of records has been selected.
Main success scenario	**[TO BE CONFIRMED: the numbered steps. Draft from Task 2 REQ-1.3 and Task 1 §3.4, feature F3.]**
Alternative flows	**[TO BE CONFIRMED: what happens when no chart type fits the question]**
Postconditions	A figure is produced and held with the exchange.
Traces to	REQ-1.3

2.3.4 UC-4: Summarise the Result in Words

Actor	System (on behalf of the Analyst)
Goal	Describe what the figure shows in one or two sentences, including the extremes found.
Preconditions	A figure exists for this exchange.
Main success scenario	**[TO BE CONFIRMED: the numbered steps. Draft from Task 2 REQ-1.4 and Task 1 §3.4, feature F4.]**
Alternative flows	**[TO BE CONFIRMED: what the system says when the summary cannot be produced]**
Postconditions	A sentence or two is shown beside the figure.
Traces to	REQ-1.4

2.3.5 UC-5: Show the Accuracy Dashboard

Actor	Reviewer or auditor
Goal	See how accurate the automatic interpretation currently is, measured against the recorded benchmark.
Preconditions	The benchmark is available.
Main success scenario	**[TO BE CONFIRMED: the numbered steps. Draft from Task 2 REQ-1.5 and Task 1 §3.4, feature F5. The accuracy figure itself is still open in Task 2 and must not be invented here.]**
Alternative flows	**[TO BE CONFIRMED: what happens when the benchmark cannot be reached]**
Postconditions	A measured accuracy is displayed beside the answers.
Traces to	REQ-1.5

2.3.6 UC-6: Record and Re-run an Exchange

Actor	Reviewer or auditor
Goal	See how a past answer was produced, and reproduce the same figure from the record alone.
Preconditions	At least one exchange has been made.
Main success scenario	**[TO BE CONFIRMED: the numbered steps. Draft from Task 2 REQ-1.6 and Task 1 §3.4, feature F6.]**
Alternative flows	**[TO BE CONFIRMED: what happens when the data behind a record has since changed]**
Postconditions	The same figure is rebuilt from the stored question, request, snapshot identifier and chart choice.
Traces to	REQ-1.6

2.4 Traceability to the Requirements

Table 2-2: Every functional requirement of Task 2 is carried by exactly one use case.

Requirement	Use case	What the use case adds to the requirement
REQ-1.1	UC-1	Names the actor, the precondition and the failure case that the requirement states only in outline.
REQ-1.2	UC-2	Fixes the snapshot identifier as the handle by which later steps refer to the selected subset.
REQ-1.3	UC-3	Records that chart choice is the system’s, not the analyst’s.
REQ-1.4	UC-4	Makes the summary a precondition of a finished exchange rather than an optional extra.
REQ-1.5	UC-5	Separates the accuracy dashboard from the question round trip, which the requirement does not do.
REQ-1.6	UC-6	States that re-running needs the record alone, not the data as it happens to stand now.

Chapter 3

Static Modelling: Class and Object Diagrams

3.1 Finding the Classes

The classes were taken from the nouns the Task 2 requirements and use case descriptions use to talk about the domain, keeping those the system must remember and dropping those that are only a way of speaking. Each class below is given a name, the attributes it holds and the operations it offers; a class with attributes but no operations would be a data record, not a class, and the analysis is not complete until the operations have been decided. *(Source: SE Chapter 5, object-oriented analysis)*

Table 3-1: Candidate classes and the reason each is kept or dropped.

Class	Where it comes from	Why it is a class
C-1 Trajectory	Movement records	The system must remember it, and it is the subject of almost every question.
C-2 Waypoint	Positions along a trajectory	A position has no meaning apart from the trajectory that contains it, which makes the relationship a composition.
C-3 MapLayer	Background map data	Layers are held and switched independently of any one trajectory.
C-4 POI	Named places	Attached to a layer; a place can be interesting to several layers.
C-5 Chart	Figures the system draws	Generalises the three chart kinds, which differ in how they are drawn but share a title, a scale and an axis convention.
C-6 Query	The analyst's question	Must be remembered so that an exchange can be re-run.
C-7 Exchange	One question and its answer	The unit the record and the re-run of UC-6 operate on.
C-8 Benchmark	Published question-and-answer pairs	Read, never written; carried as a class so that UC-5 has something to run against.
C-9 AccuracyReport	The measured accuracy	What UC-5 displays.

[TO BE CONFIRMED: the classes that the assigned customer's own data model implies, once Part A source 1 has been elicited]

3.2 Class Diagram

Figures 3-1 and 3-2 show the classes with their attributes and their operations, and the relationships between them. They are drawn as two diagrams rather than one: a single diagram holding all twelve classes comes out about 1900 pixels wide, which at the width of the text block scales the labels below 6 pt and makes them unreadable, whereas each half stays above 9 pt. Figure 3-1 holds the data classes and Figure 3-2 the classes that carry a question through to a recorded answer.

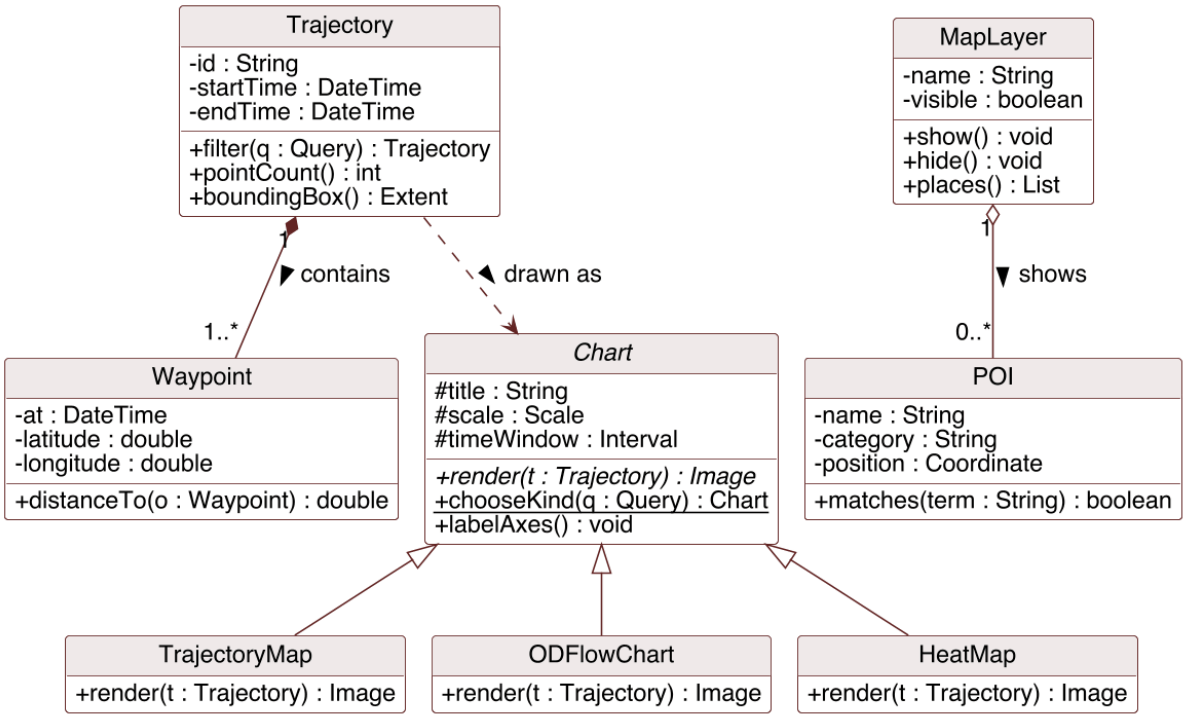


Figure 3-1: Class diagram, part 1: the data classes.

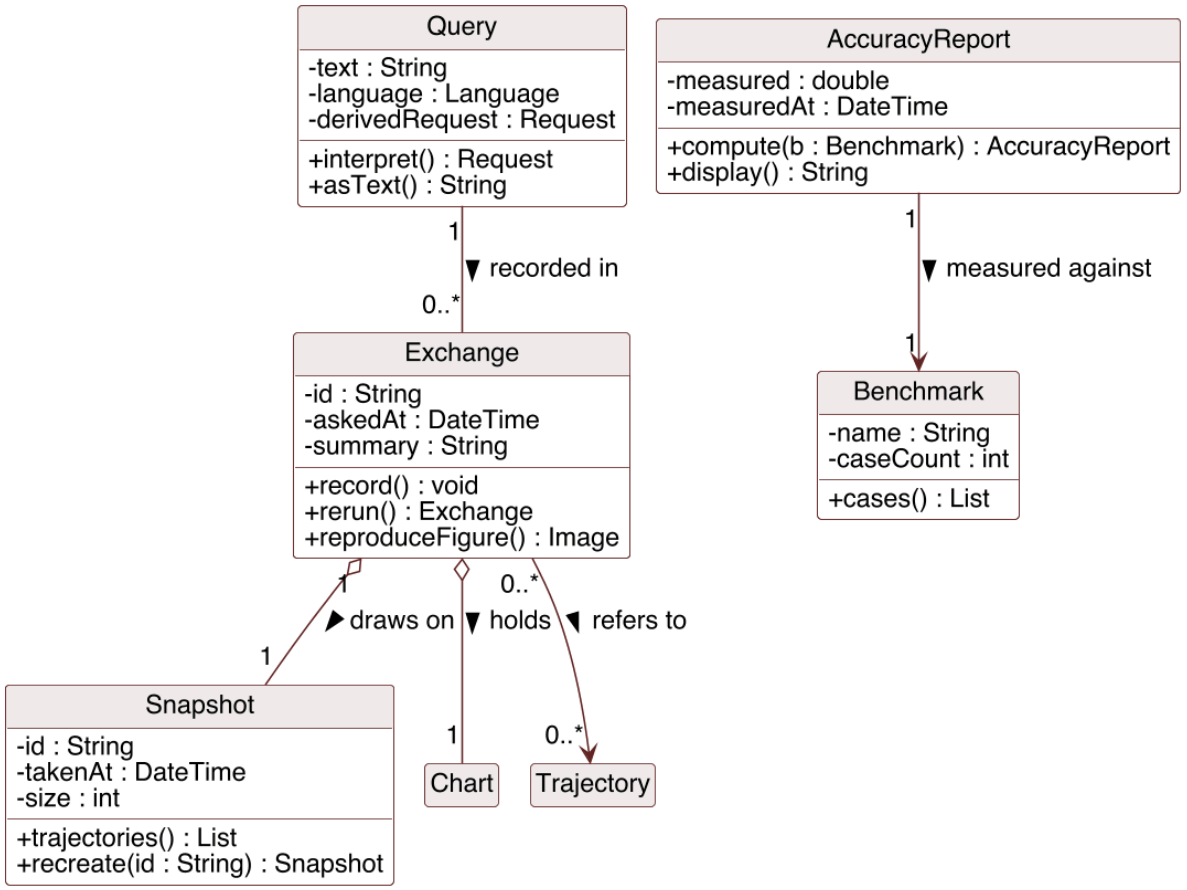


Figure 3-2: Class diagram, part 2: the question-and-answer classes.

3.3 Relationships

Table 3-2: Relationships between the classes, read from Figures 3-1 and 3-2.

Whole	Part	Kind	Why
Trajectory	Waypoint	Composition	A waypoint has no meaning apart from its trajectory; deleting the trajectory deletes its waypoints.
MapLayer	POI	Aggregation	A place can outlive the layer it is currently shown on.
Chart	TrajectoryMap, ODFlowChart, HeatMap	Generalisation	The three differ only in how they draw; the shared parts belong in the parent.
Exchange	Query, Chart	Aggregation	An exchange is built from a question and the figure that answered it.
AccuracyReport	Benchmark	Association	An accuracy figure is only meaningful against the benchmark it was measured on.

3.4 Object Diagram

Part A of the task description names the object diagram as the static model to be provided. A class diagram says what kinds of thing may exist; an object diagram says what exists at one moment, with the links between the individual objects shown. Figure 3-3 is the state of the system immediately after one question has been answered, which is the moment that makes every class in Figures 3-1 and 3-2 visible at once.

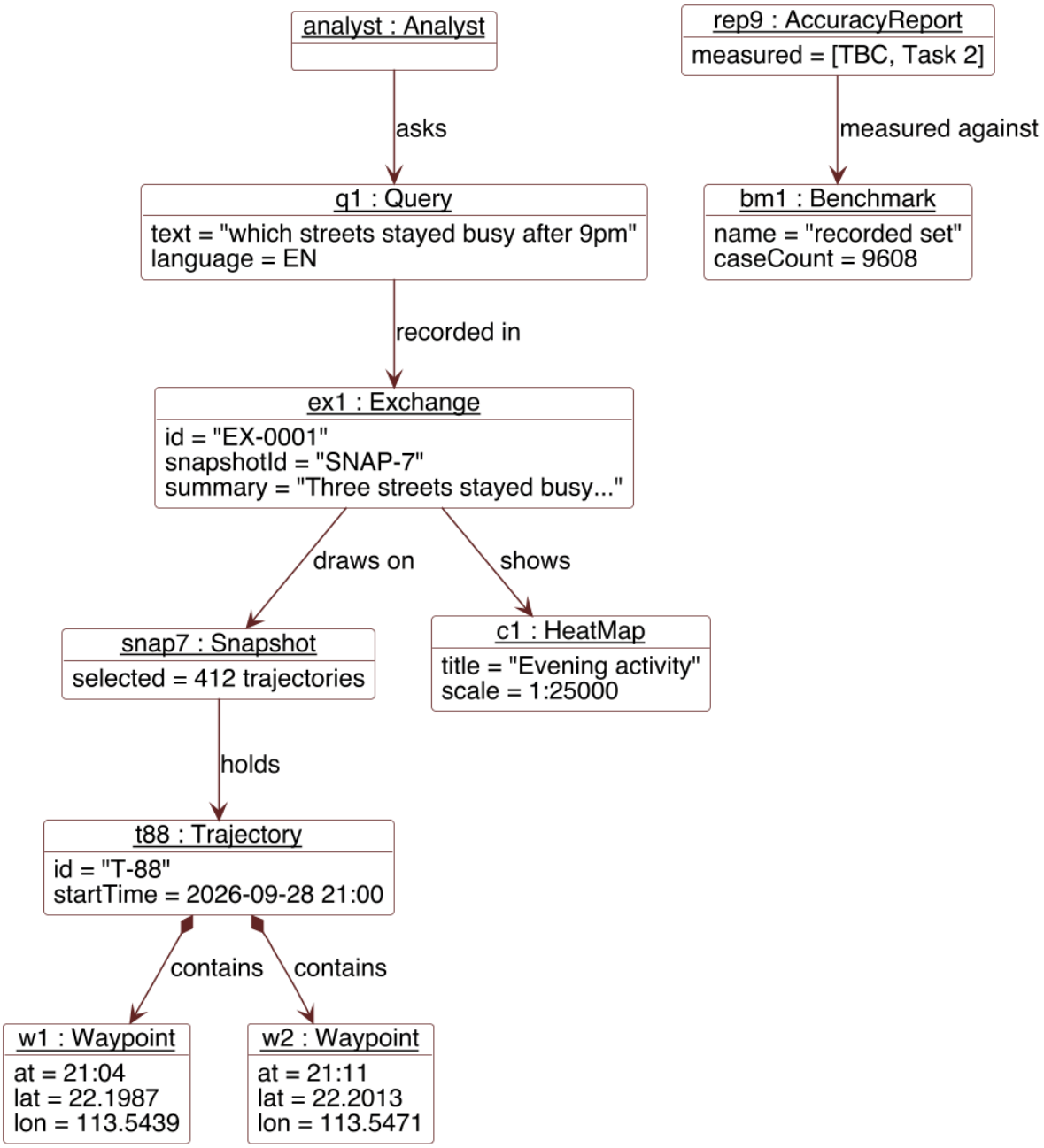


Figure 3-3: Object diagram for one answered exchange.

The two diagrams are consistent by construction: every object in Figure 3-3 is an instance of exactly one class in Figures 3-1 and 3-2, every link between objects is an instance of one relationship in Table 3-2, and no object appears that has no class.

3.5 Traceability to the Requirements

Table 3-3: Classes, and the requirements they carry out.

Class	Requirement	Operation that carries it out
C-6 Query	REQ-1.1	Interpreting a question written in ordinary language into an exact request.
C-1 Trajectory	REQ-1.2	Filtering the stored trajectories against a written request.
C-5 Chart	REQ-1.3	Choosing the chart kind that fits the question and rendering it.
C-7 Exchange	REQ-1.4, REQ-1.6	Holding the summary with the figure, and re-running the exchange from the record alone.
C-8 Benchmark, C-9 AccuracyReport	REQ-1.5	Re-running the recorded benchmark and reporting the measured accuracy.

Chapter 4

Dynamic Modelling: Sequence Diagrams

This chapter gives one sequence diagram for each of the six use cases of Chapter 2, as Part A of the task description requires. A sequence diagram shows the messages that pass between the participants while the use case runs, in the order in which they pass, which is the dynamic counterpart of the class diagram: the classes say who may exchange messages, the sequence diagrams say in what order they do.

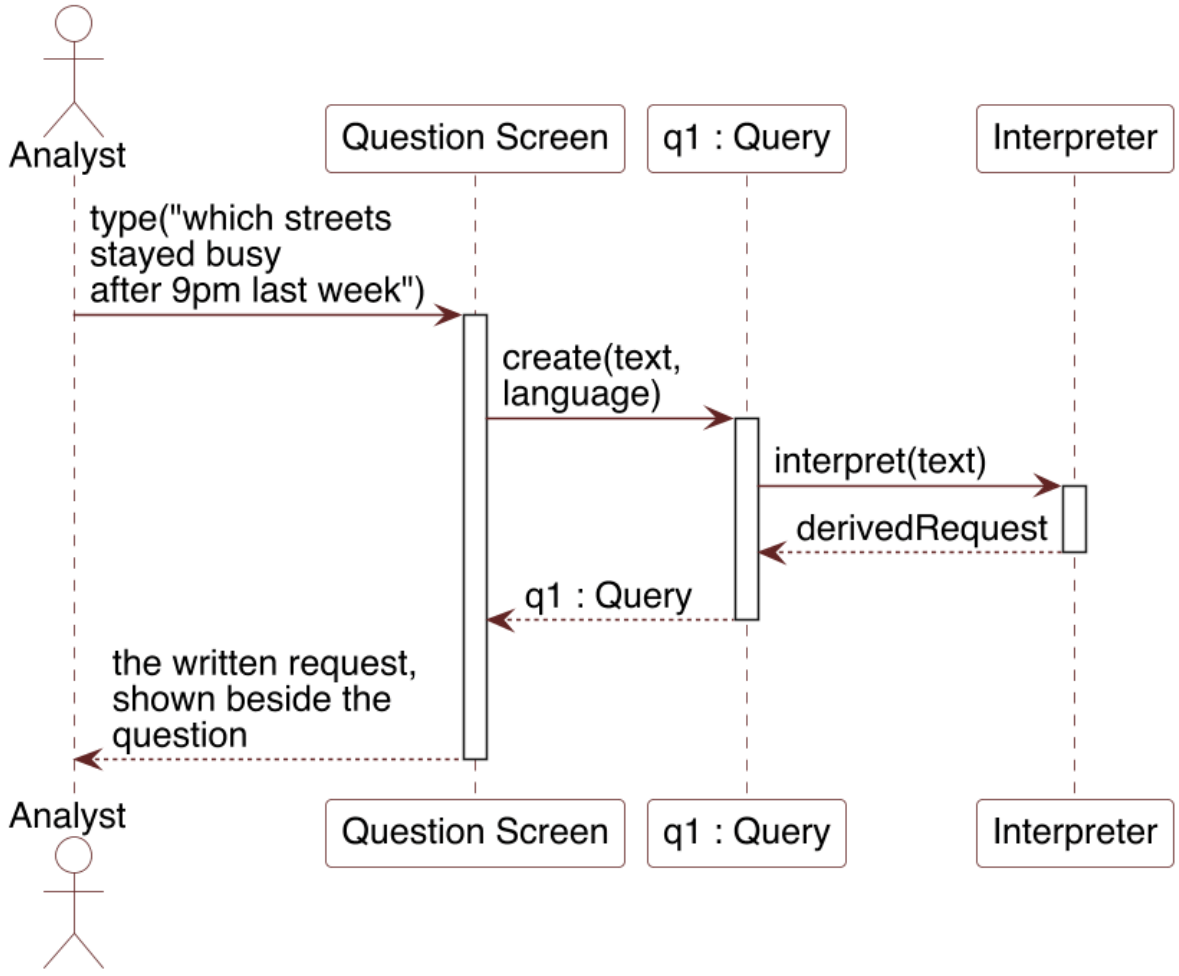


Figure 4-1: Sequence diagram for UC-1.

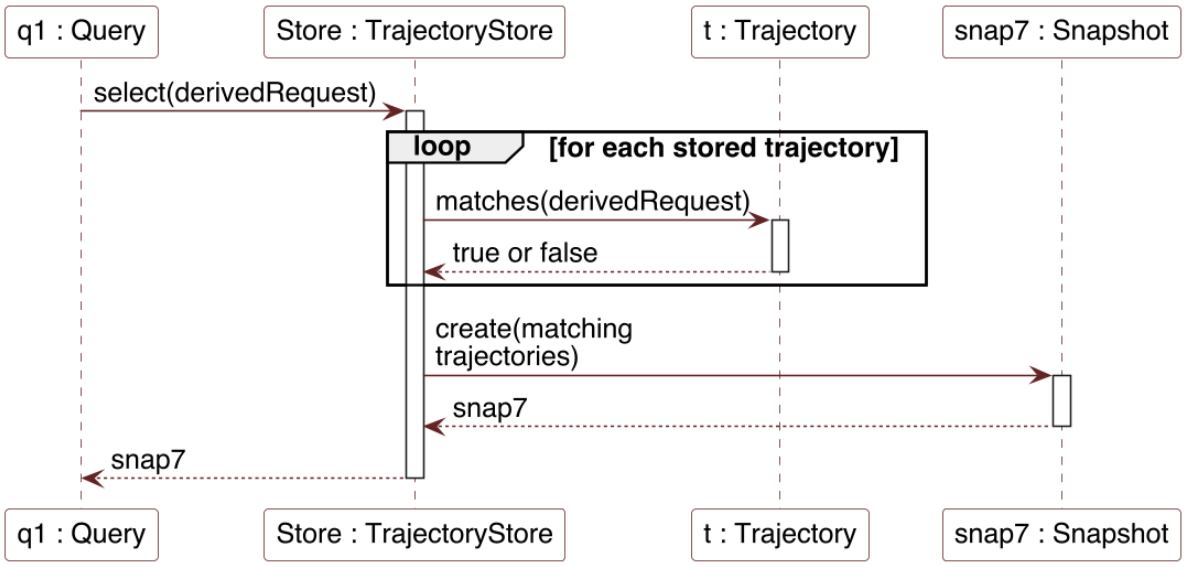


Figure 4-2: Sequence diagram for UC-2.

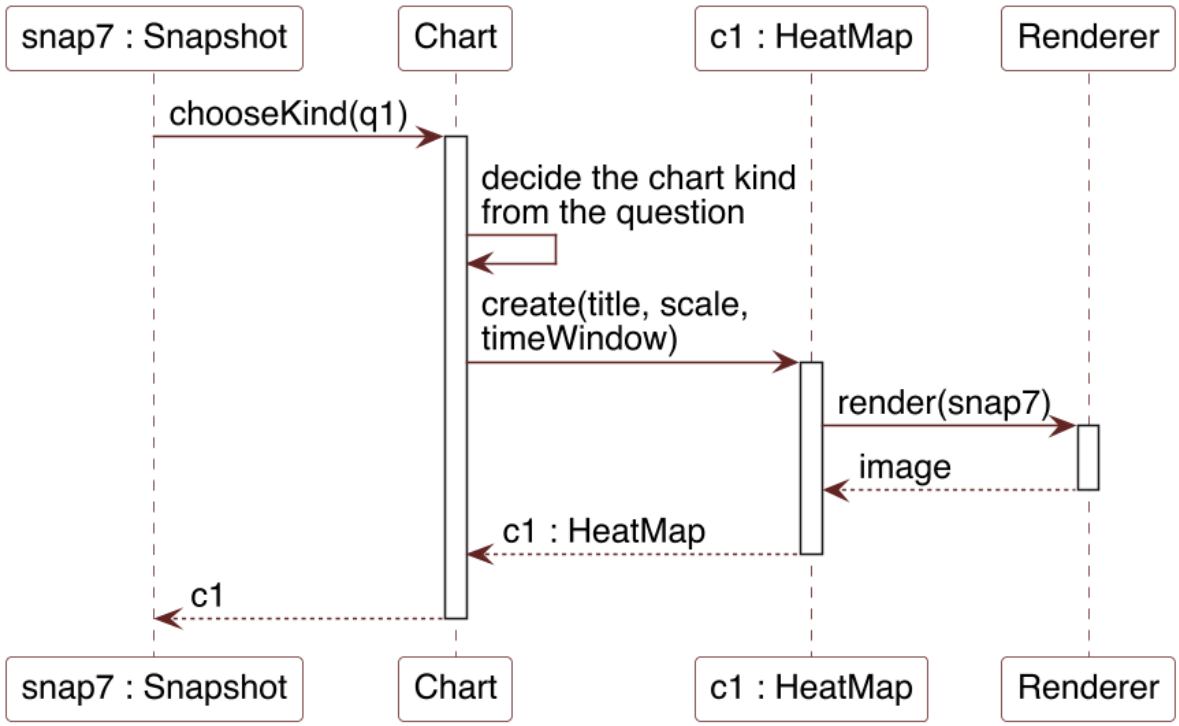


Figure 4-3: Sequence diagram for UC-3.

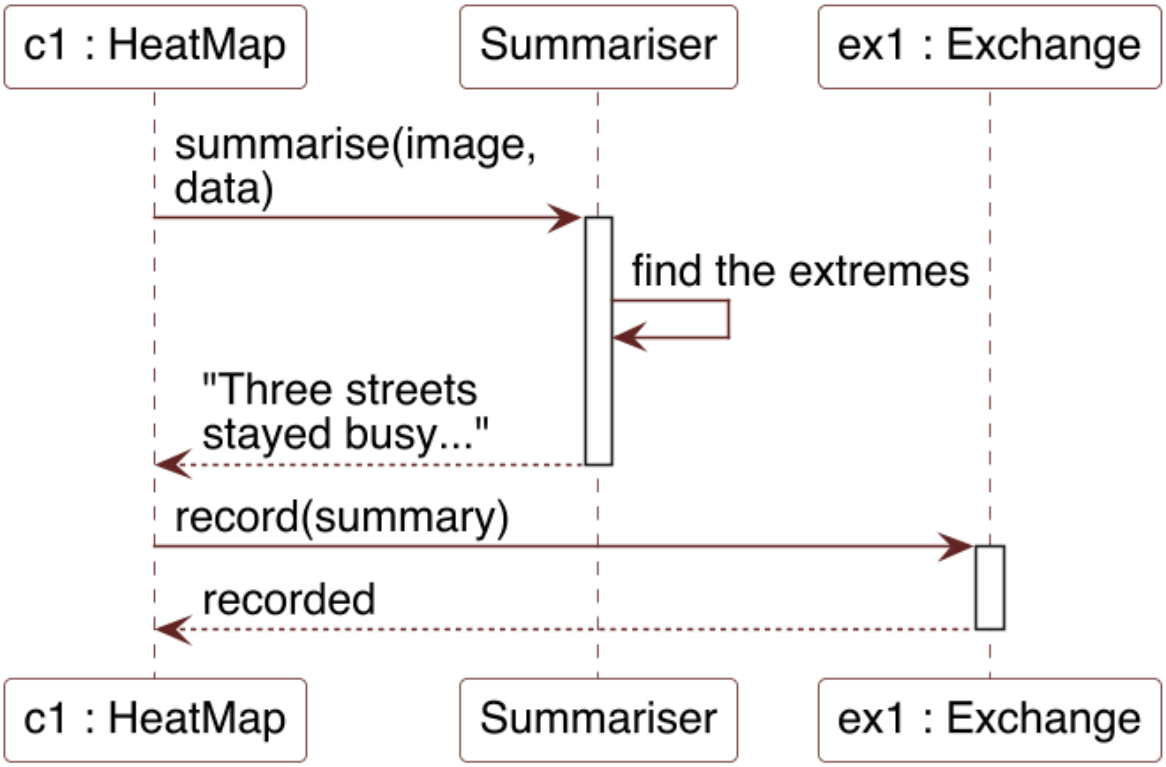


Figure 4-4: Sequence diagram for UC-4.

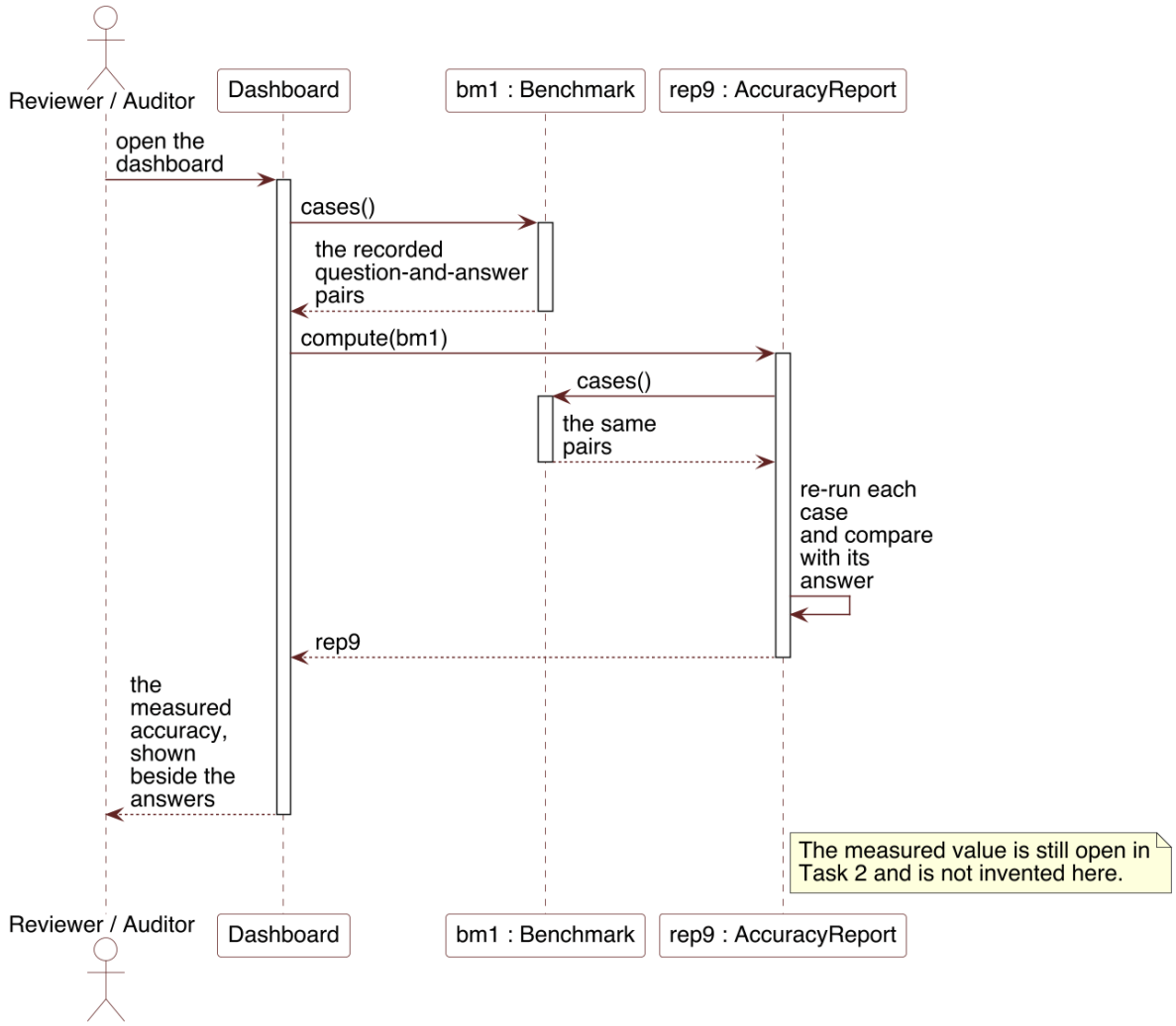


Figure 4-5: Sequence diagram for UC-5.

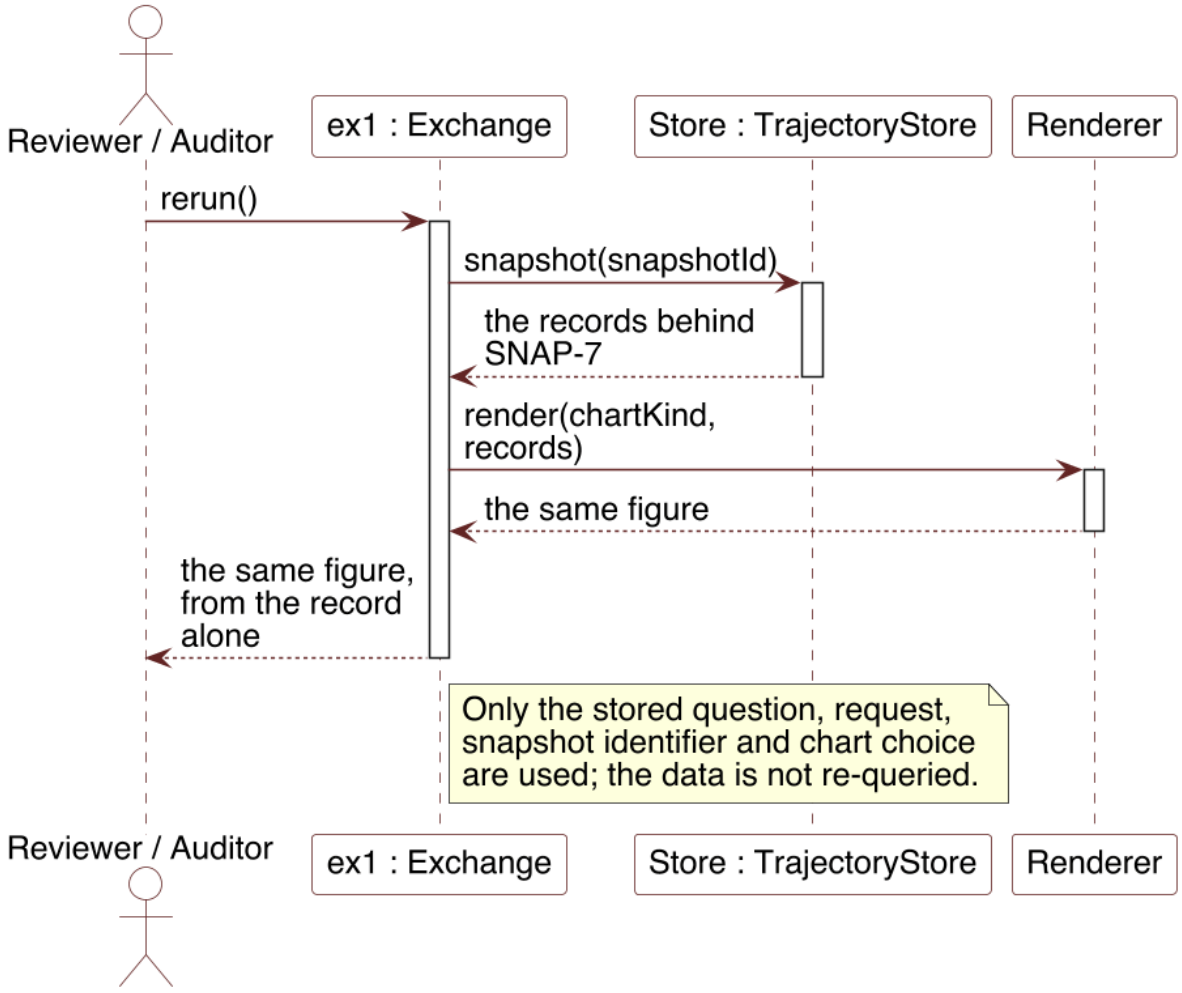


Figure 4-6: Sequence diagram for UC-6.

4.1 Consistency with the Class Diagram

Every message in Figures 4-1 to 4-6 is sent to a lifeline whose type is a class in Figures 3-1 and 3-2, and every message name is an operation of that class. Conversely, every operation in Figures 3-1 and 3-2 appears in at least one sequence diagram; an operation that no use case calls would be a class member with no requirement behind it.

Chapter 5

Behavioural Modelling: State Diagram

Part A(4) of the task description welcomes further diagrams beyond the three it requires. The one added here is a state diagram, because the model contains exactly one object whose behaviour depends on the state it is in rather than only on the messages it receives: an exchange, which Task 2's REQ-1.6 requires to be re-runnable after it is finished, and which may be re-run more than once.



Figure 5-1: State diagram for an exchange.

The transition from the finished state back to the re-run state is what makes the exchange re-runnable rather than a record that can only be read. **[TO BE CONFIRMED: any further state-dependent object the assigned customer's own processes imply, once Part A source 1 has been elicited]**

References

[1] Group 1, *TrajVis Studio: A Self-Service Platform for Natural-Language Trajectory Analysis*, Task 1 project proposal, Software Engineering, MUST, 2026.

[2] Group 1, *TrajVis Studio: User Requirements Specification*, Task 2 deliverable, Software Engineering, MUST, 2026.

[3] *Requirements Analysis*, Software Engineering, MUST, 2026.

[4] *Surface for Requirements Analysis*, template supplied with Task 3, Software Engineering, MUST, 2026.

[5] Project repository, https://github.com/chenhongzhou666/MUST-SE-TrajVis

[6] *SE Chapter 5: Requirements Analysis II (Object-Oriented Analysis)*, lecture slides, Software Engineering, MUST, 2026.09.30.